Department of Electrical & Electronics Engineering
Abdullah Gül University

EE 421 Geometrical Optics

Phase II Report

Electronic Design and Control

Submitted on: 15.05.2026
Submitted by: Sude Nur Görez 2111011026

Lab Partners:
Yusuf Taha Gümüş 2111011025
Yusuf Yıldız 2111011006

1. Objective

Phase II focuses on the electronic and software design of the galvo scanner system. In Phase I, we modeled the optical layout in Zemax and found that our two-mirror system can cover a scan area of approximately 142 mm × 140 mm at a throw distance of 500 mm. Mirror 1 controls the horizontal axis (Tilt About Y, neutral 135°, range ~7°) and Mirror 2 controls the vertical axis (Tilt About X, neutral 0°, range ~19°).

In this phase, we select the motors, microcontroller, and laser module; design the driving circuits; develop the control software; and encode at least five letters as scan trajectories.

2. System Block Diagram

The system has four subsystems: a USB power bank for power, an Arduino Uno as the controller, two servo motors for mirror positioning, and a transistor-switched laser module. The Arduino generates PWM signals to position the servos and a digital signal to turn the laser on and off.

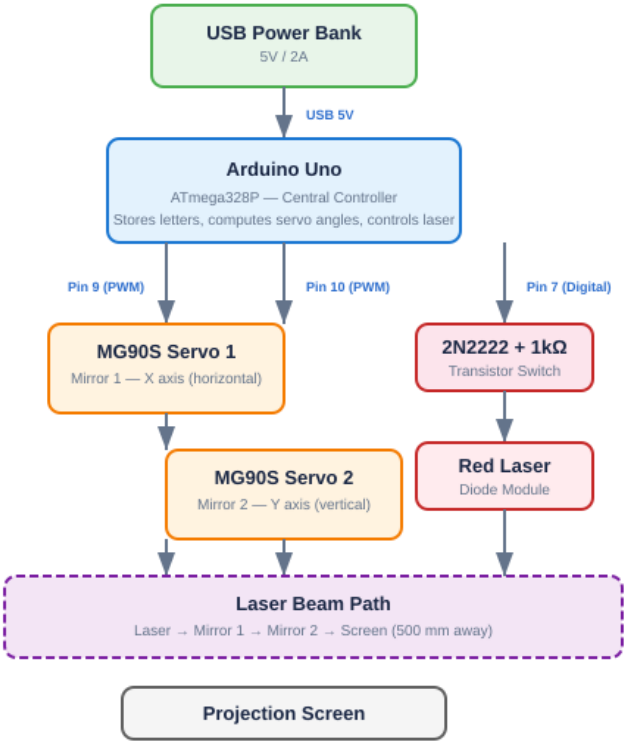


Figure 1. System block diagram showing signal and power flow.

3. Motor Selection

3.1 Comparison of Motor Types

We considered four motor types before choosing. The table below summarizes the key differences:

Criterion	MG90S Servo	28BYJ-48 Step	DC Motor+Encoder	Galvanometer
Speed	0.10 s/60° (fast)	~15 RPM (very slow)	Fast	Very fast (<1 ms)
Control	PWM, easy	Needs step driver	PID + encoder	DAC + amplifier
Backlash	<1° (metal gears)	<0.1°	Depends on gearing	None
Complexity	Low	Medium	High	High
Verdict	**SELECTED**	Rejected	Rejected	Rejected

3.2 Why MG90S?

The 28BYJ-48 stepper motor was considered because it is cheap and already available. However, it requires an external ULN2003 driver, produces discrete stepwise motion, and makes the control software more complex than a directly controlled servo. For this project, the MG90S servo is more practical because it has built-in position feedback, can be driven directly using the Arduino Servo library, and is easier to calibrate for point-to-point vector drawing.

The MG90S has metal gears (less backlash than the plastic-geared SG90), a dead band of ~5 µs, and moves at 0.10 s per 60°. It connects directly to the Arduino with no extra driver board. The 180° range is more than enough for our ~7° and ~19° scan needs.

We chose the MG90S over the basic SG90 specifically because the MG90S's smaller dead band (~5 µs vs ~10 µs) gives us roughly double the positional resolution on the X axis. Since Mirror 1's scan only spans 7° (about 74 µs of servo travel), this difference matters a lot for drawing smooth horizontal lines.

4. Microcontroller

We chose the Arduino Uno because it runs at 5 V (matches the servos directly), the Servo.h library handles PWM generation automatically, and everyone in our group has used it before. We only need 3 pins: Pin 9 for Servo 1, Pin 10 for Servo 2, and Pin 7 for the laser switch.

An ESP32 would also work but outputs 3.3 V, which might cause servo jitter without a level shifter. Since we already had access to an Arduino Uno, there was no reason to complicate things.

5. Laser Driver Circuit

A 5 V red laser diode module with a built-in current-limiting resistor will be used. The selected module is expected to operate in the visible red range, typically around 650 nm, and its optical output power will be kept below 5 mW for safety. Before Phase III assembly, the module voltage and current rating will be verified from the supplier label or by measurement.

To switch the laser from software, we use a 2N2222 NPN transistor as a simple on/off switch. When Arduino Pin 7 goes HIGH, current flows through the 1 kΩ base resistor into the transistor, saturating it and turning the laser on. When Pin 7 goes LOW, the transistor cuts off and the laser turns off.

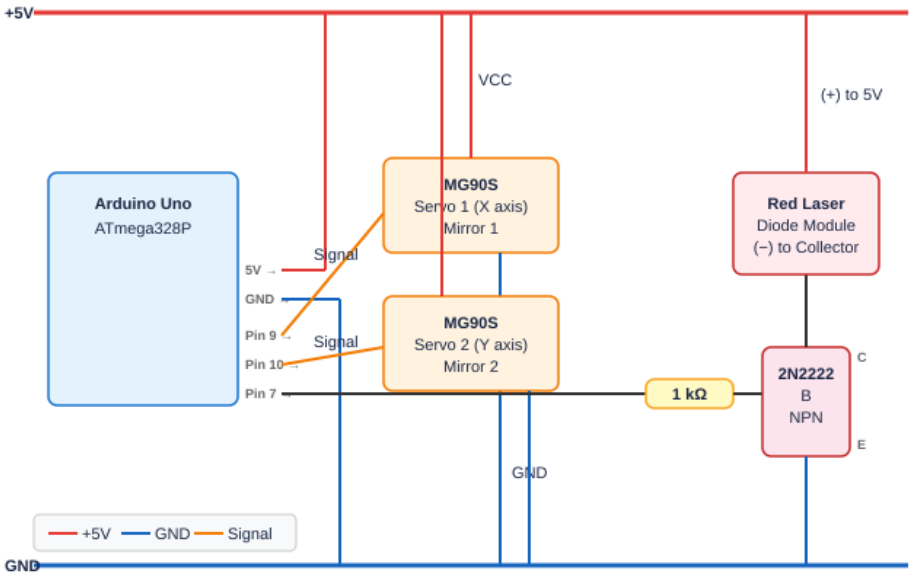


Figure 2. Complete circuit diagram. Red lines are +5V, blue lines are GND, orange lines are signal wires.

6. Power Requirements

The USB power bank provides the 5 V supply. The servos and laser module are powered from the 5 V breadboard rail, while the Arduino shares the same ground reference. The servo current should not be routed through an Arduino I/O pin. A common ground between the Arduino, servos, transistor circuit, and laser module is required for stable signal reference. The current budget is:

Component	Voltage	Typical (mA)	Peak (mA)
Arduino Uno	5 V	~50	~50
MG90S Servo 1	5 V	~100	~650
MG90S Servo 2	5 V	~100	~650
Laser	5 V	~25	~30
TOTAL	5 V	~275	~1380

The typical draw of ~275 mA is well under the 2 A capacity of a standard power bank. If servo jitter occurs due to current spikes, a 470 µF or 1000 µF electrolytic capacitor can be added across the servo 5 V and GND rails.

7. Control Logic

We use point-to-point vector drawing. Each letter is stored as a list of (X, Y) target points on the screen and a sequence of line segments to connect them. The algorithm is:

1. Turn the laser off and move the servos to the starting point of the first stroke.
2. Wait ~18 ms for the servos to settle.
3. If this segment is a drawing stroke, turn the laser on.

4. Interpolate from start to end in 25 small steps, updating both servos at each step with a ~12 ms delay. This traces a smooth line on the screen.
5. Turn the laser off at the end of the segment.
6. If the next stroke is disconnected (like the crossbar of H), move with laser off. Otherwise continue drawing.
7. After all strokes, pause 600 ms before the next letter.

We chose vector drawing over raster scanning because the MG90S servos are too slow for raster. With vector drawing, each letter takes about 1 second, which is slow but recognizable.

8. Mirror Angle to Servo Mapping

From Phase I, we established these calibration relationships:

- Mirror 1: 0.0492° per mm (X axis). Decreasing the tilt from 135° moves the beam to the right.
- Mirror 2: 0.1139° per mm (Y axis). A negative tilt moves the beam upward.

To convert a desired screen position (X, Y) to servo pulse widths, we use:

Servo 1 (µs) = 1450 – 0.519 × X_mm

Servo 2 (µs) = 1450 – 1.203 × Y_mm

The value 1450 µs is our starting estimate for servo center. The constants 0.519 and 1.203 come from combining the Phase I angle-per-mm factors with the servo's 10.56 µs/° resolution. These will all be fine-tuned during assembly.

9. Letter Encoding

We encoded seven letters: H, E, L, N, O, T, and A. Each table shows the target coordinates, computed mirror angles, and servo pulse widths. The letter N was already verified in Phase I.

Letter: H

Point	X(mm)	Y(mm)	M1 TiltY(°)	M2 TiltX(°)	Servo1(µs)	Servo2(µs)
H1	-30	-40	136.47	4.55	1466	1498
H2	-30	40	136.47	-4.55	1466	1402
H3	-30	0	136.47	0.00	1466	1450
H4	30	0	133.53	0.00	1434	1450
H5	30	-40	133.53	4.55	1434	1498
H6	30	40	133.53	-4.55	1434	1402

Drawing sequence: H1→H2 (left vertical, ON), H3→H4 (horizontal bar, ON), H5→H6 (right vertical, ON)

Letter: E

Point	X(mm)	Y(mm)	M1 TiltY(°)	M2 TiltX(°)	Servo1(µs)	Servo2(µs)
E1	-30	-40	136.47	4.55	1466	1498
E2	-30	40	136.47	-4.55	1466	1402
E3	30	40	133.53	-4.55	1434	1402
E4	-30	0	136.47	0.00	1466	1450

E5	20	0	134.02	0.00	1440	1450
E6	30	-40	133.53	4.55	1434	1498

Drawing sequence: E1→E2 (left vertical, ON), E2→E3 (top bar, ON), E4→E5 (middle bar, ON), E1→E6 (bottom bar, ON)

Letter: L

Point	X(mm)	Y(mm)	M1 TiltY(°)	M2 TiltX(°)	Servo1(µs)	Servo2(µs)
L1	-30	40	136.47	-4.55	1466	1402
L2	-30	-40	136.47	4.55	1466	1498
L3	30	-40	133.53	4.55	1434	1498

Drawing sequence: L1→L2 (vertical, ON), L2→L3 (bottom bar, ON)

Letter: N

Point	X(mm)	Y(mm)	M1 TiltY(°)	M2 TiltX(°)	Servo1(µs)	Servo2(µs)
N1	-35	-45	136.72	5.12	1468	1504
N2	-35	45	136.72	-5.12	1468	1396
N3	35	-45	133.28	5.12	1432	1504
N4	35	45	133.28	-5.12	1432	1396

Drawing sequence: N1→N2 (left vertical, ON), N2→N3 (diagonal, ON), N3→N4 (right vertical, ON)

Letter: O

Point	X(mm)	Y(mm)	M1 TiltY(°)	M2 TiltX(°)	Servo1(µs)	Servo2(µs)
O1	0	40	135.00	-4.55	1450	1402
O2	-30	20	136.47	-2.28	1466	1426
O3	-30	-20	136.47	2.28	1466	1474
O4	0	-40	135.00	4.55	1450	1498
O5	30	-20	133.53	2.28	1434	1474
O6	30	20	133.53	-2.28	1434	1426

Drawing sequence: O1→O2→O3→O4→O5→O6→O1 (closed loop, ON)

Letter: T

Point	X(mm)	Y(mm)	M1 TiltY(°)	M2 TiltX(°)	Servo1(µs)	Servo2(µs)
T1	-30	40	136.47	-4.55	1466	1402
T2	30	40	133.53	-4.55	1434	1402
T3	0	40	135.00	-4.55	1450	1402
T4	0	-40	135.00	4.55	1450	1498

Drawing sequence: T1→T2 (top bar, ON), T3→T4 (vertical stem, ON)

Letter: A

Point	X(mm)	Y(mm)	M1 TiltY(°)	M2 TiltX(°)	Servo1(µs)	Servo2(µs)
A1	-30	-40	136.47	4.55	1466	1498
A2	0	40	135.00	-4.55	1450	1402
A3	30	-40	133.53	4.55	1434	1498

A4	-15	0	135.74	0.00	1458	1450
A5	15	0	134.26	0.00	1442	1450

Drawing sequence: A1→A2 (left diagonal, ON), A2→A3 (right diagonal, ON), A4→A5 (horizontal bar, ON)

All letters fit within ±35 mm (X) and ±45 mm (Y), well inside the 142 × 140 mm scan area. The tallest letter (N, 90 mm) is 18 times the ~5 mm beam diameter, so every stroke should be clearly distinguishable.

10. Software Flowchart

The flowchart below shows the complete program flow. The software first initializes the hardware, then enters a main loop where it draws each letter by iterating through its segments. Each segment is either a drawing stroke (laser on) or a repositioning move (laser off). The interpolation engine divides each stroke into 25 steps for smooth lines.

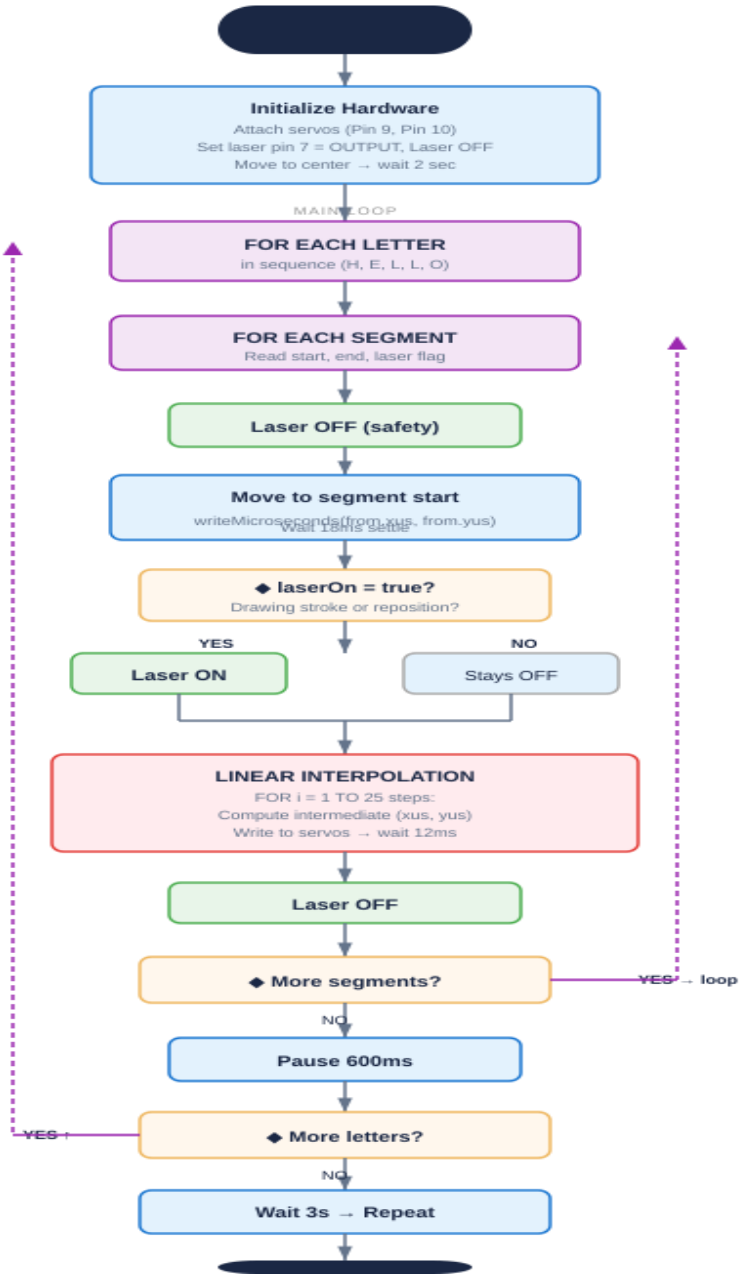


Figure 3. Software flowchart. Purple boxes are loop structures, green boxes are laser control, red box is the interpolation engine, and yellow boxes are decision points.

11. Safety Considerations

The design assumes a red laser module with optical output power below 5 mW. The module must be verified to be below this limit before testing. Even at this power level, direct and mirror-reflected eye exposure must be avoided. The system will only be operated with the projection screen in place, and the beam path will always terminate on the screen surface. The software defaults to laser-off at startup and at the end of every segment. A

manual toggle switch will also be added in series with the laser power line so the laser can be shut off instantly if anything goes wrong.

12. Potential Problems and Solutions

Problem	Cause	Solution
Servo jitter	Voltage drops on 5V rail	Add 470 µF or 1000 µF capacitor across servo power rail, or use separate 5 V supply for servos
Backlash errors	Gear play in servo gearbox	Approach from same direction; increase steps
Mirror detachment	Hot glue weakened by vibration	Use hot glue and double-sided tape
Laser not visible	Ambient light too strong	Dim the room or switch to green laser
Calibration mismatch	Physical throw distance differs from Zemax model	Software constants (center, scale) are adjustable
Letters drawn too slowly	Interpolation step delay too long	Reduce STEP_DELAY and STEPS_PER_SEG in code

13.Materials

#	Component	Qty
1	Arduino Uno	1
2	MG90S servo motor	2
3	Red laser diode module	1
4	2N2222A NPN transistor	1
5	1 kΩ resistor	1
6	Breadboard	1
7	Jumper wire set	1
8	USB cable (A-to-B)	1
9	USB power bank	1
10	Flat mirror (~25 mm)	2
11	Hot glue gun	1
12	White poster board (screen)	1
13	470 µF electrolytic capacitor	1
14	Manual laser toggle switch	1

14. Conclusion

In this report we designed the electronic control system for our two-axis galvo scanner. We selected MG90S servo motors for mirror actuation, an Arduino Uno as the controller, and a transistor switching circuit for laser control. Seven letters were encoded as scan trajectories using the Phase I Zemax calibration data.

The system is powered by a USB power bank at ~275 mA typical draw. All calibration constants in the software are adjustable, so we can fine-tune the system during Phase III assembly. The design is ready for physical implementation.

References

[1] EE 421 Geometrical Optics – Semester Project, Spring 2026.

[2] Phase I Report: Optical Design of the Two-Axis Galvo Scanner, 03.05.2026.

[3] MG90S Micro Servo Motor Datasheet.

[4] ATmega328P Microcontroller Datasheet.